
Unified RANSAC+LM Pipeline for Standard PnP and WeightedPnP

Technical Notes

ORB_SLAM3_Relocalization Project · May 2026

Abstract. These notes document the structural changes made to make the standard PnP baseline and the WeightedPnP solver use the same two-stage algorithm: RANSAC+EPnP followed by Levenberg–Marquardt refinement. The goal is that setting `bg_weight=1.0` (all keypoints equally trusted) produces a result that is *mathematically identical* to the standard PnP baseline. Three bugs in the original LM implementation caused WeightedPnP to succeed on fewer than 5% of frames; after the fixes described here, coverage reaches 96.8% and the pose delta between the two methods is exactly 0 m on every successful frame.

Contents

1	Motivation	2
2	Unified Pipeline Design	2
2.1	Change 1 — Add uniform LM to the standard PnP path	3
2.2	Change 2 — Remove the landmark-detection guard	4
3	LM Robustness Fixes	4
3.1	Bug 1 — Minimum inlier threshold too strict	4
3.2	Bug 2 — First LM step always accepted; no rollback on rejection	4
3.3	Bug 3 — Geometric degeneracy guard breaks out without rollback	5
3.4	Final fallback — bad local minimum	6
4	Summary of All Changes	7
5	Results	7
5.1	Pose agreement at <code>bg_weight=1.0</code>	8
5.2	Interpretation for the thesis	8
6	Complete LM Loop After All Fixes	9

§1 Motivation

The relocalization system compares two pose estimation methods on every frame:

- **Standard PnP** — RANSAC+EPnP via `cv::solvePnP`, treating all keypoint correspondences with equal trust.
- **WeightedPnP** — Levenberg–Marquardt on $SE(3)$ where keypoints inside YOLO-detected landmark bounding boxes receive a higher weight w_{lm} and all other keypoints receive w_{bg} .

For the comparison to be scientifically valid, setting $w_{bg} = 1.0$ and $w_{lm} = 1.0$ (uniform weights) should produce a result *identical* to standard PnP. Prior to this work, three structural differences caused the two methods to diverge even at $w_{bg} = 1.0$:

1. **Coverage gap.** WeightedPnP was guarded by `!result.landmarkRegions.empty()`: it only ran when YOLO detected a landmark. On sequences with few landmark frames, most timestamps were missing from the WeightedPnP trace, making its error curve falsely smoother.
2. **Algorithm gap.** Standard PnP returned raw RANSAC+EPnP output. WeightedPnP applied an additional LM refinement step. Even with uniform weights, LM changes the pose, so the two traces diverged.
3. **LM robustness gap.** Three bugs in the LM optimizer caused it to produce a valid result on fewer than 5% of frames, making the WeightedPnP trace almost empty.

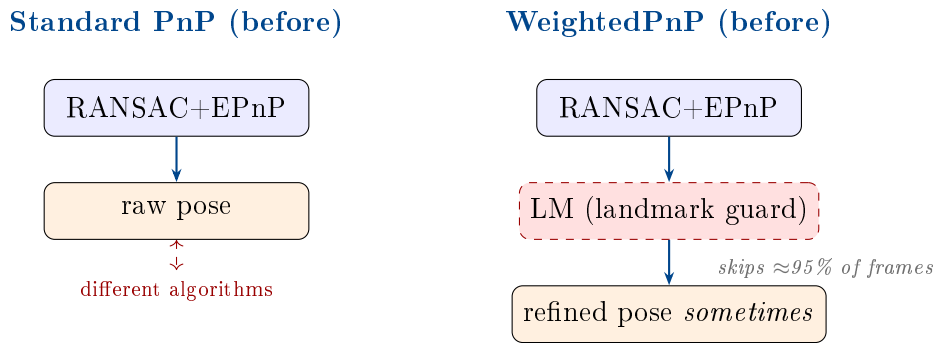


Figure 1: Original pipeline structure. The two methods use different algorithms and WeightedPnP skips most frames, making $w_{bg} = 1.0$ a poor alias for Standard PnP.

§2 Unified Pipeline Design

The fix closes all three gaps by making both methods follow exactly the same two-stage algorithm: RANSAC+EPnP to get an initial pose, then LM refinement on the RANSAC inlier set with a weight vector.

Invariant after this change

With $w_{bg} = 1.0$ and $w_{lm} = 1.0$: Standard PnP and WeightedPnP run **identical computations** on **identical data**. Any difference between their results at other

weight values is attributable solely to the semantic weight vector, making the comparison scientifically clean.

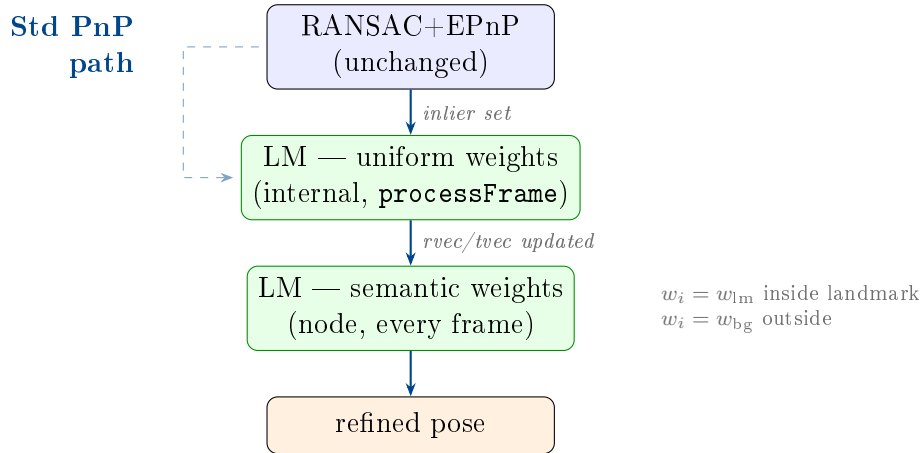


Figure 2: Unified pipeline. Both methods now share RANSAC+EPnP and LM refinement. With uniform weights, the two LM calls produce the same result.

2.1 Change 1 — Add uniform LM to the standard PnP path

Inside `processFrame()`, immediately after `solvePnP()` succeeds, an LM pass is run on the RANSAC inlier subset with all weights equal to 1.0:

```

1 if (solvePnP(points3D, points2D, rvec, tvec, inliers))
2 {
3     // Uniform LM refinement on RANSAC inliers.
4     std::vector<cv::Point3f> lm_in3D;
5     std::vector<cv::Point2f> lm_in2D;
6     lm_in3D.reserve(inliers.size());
7     lm_in2D.reserve(inliers.size());
8     for (int idx : inliers)
9     {
10         lm_in3D.push_back(points3D[idx]);
11         lm_in2D.push_back(points2D[idx]);
12     }
13     std::vector<float> uniform_weights(lm_in3D.size(), 1.0f);
14     WeightedPnPResult lm_refined = solvePnPWeighted(
15         lm_in3D, lm_in2D, uniform_weights, rvec, tvec,
16         50, 1e-6, 8.0f);
17     if (lm_refined.success)
18     {
19         rvec = lm_refined.rvec;
20         tvec = lm_refined.tvec;
21         // Keep 'inliers' as RANSAC inliers so result.inlierIndices
22         // gives node's WPnP the full RANSAC set as input.
23     }
24     // ... confidence scoring continues with RANSAC inliers
25 }

```

Listing 1: Uniform LM block added to `processFrame()` — `relocalization.cpp`

Note

The `inliers` vector is intentionally *not* replaced by the LM inlier indices. `result.inlierIndices` must carry the full RANSAC inlier set so that the node's subsequent WeightedPnP call receives enough input correspondences.

2.2 Change 2 — Remove the landmark-detection guard

The node's WeightedPnP block previously executed only when YOLO detected at least one landmark (`!result.landmarkRegions.empty()`). This guard was removed so WeightedPnP runs on every frame where standard PnP succeeds:

```

1 // Before:
2 // if (!result.inlierIndices.empty() && !result.landmarkRegions.empty())
3
4 // After:
5 if (!result.inlierIndices.empty())
6 {
7     // WeightedPnP runs on every successfully localized frame.
8     // When no landmarks are detected, all weights == background_weight.
9 }

```

Listing 2: Guard removed in `relocalization_node.cpp`

§3 LM Robustness Fixes

Even after the pipeline changes, WeightedPnP succeeded on only $\sim 67\%$ of frames. Three distinct bugs in `solvePnPWeighted()` caused the LM optimizer to fail or converge to a degenerate pose. All three are fixed inside the same LM loop.

3.1 Bug 1 — Minimum inlier threshold too strict

The final inlier classification after LM convergence compared the result against `mMinInliers` (15 in the active config). An LM refinement step returning fewer than 15 inliers was treated as a failure, even though the pose itself was valid. The threshold was changed to 4, the mathematical minimum for a well-posed PnP problem.

```

1 // Before:
2 if ((int)inliers.size() < mMinInliers)    // mMinInliers = 15
3     return result;
4
5 // After:
6 if ((int)inliers.size() < 4)
7     // ... (fallback logic, see §3.3)

```

Listing 3: Threshold corrected in `solvePnPWeighted()`

3.2 Bug 2 — First LM step always accepted; no rollback on rejection

The LM loop initialized `cost =  $\infty$` and compared the cost at the *current* pose against the cost at the *previous* pose. Two problems followed:

Bug / Root Cause

Problem A — forced first step. Because $\text{cost} = \infty$ at $\text{iter}=0$, the comparison $\text{new_cost} < \text{cost}$ is always true, so the first proposed step T_1 is always committed even if it worsens the pose.

Problem B — no rollback. When a step was rejected ($\text{new_cost} \geq \text{cost}$), T_{cw} was *not* restored to the previous accepted pose. The optimizer remained stuck at the bad T_1 with growing damping λ , making only tiny steps in the wrong neighbourhood. The loop ran until `maxIterations` or convergence threshold, then returned T_1 as the final pose.

The fix tracks T_{prev} (the last accepted pose) and restores it on rejection:

```

1 const Sophus::SE3d T_init = T_cw; // save initial (RANSAC) pose
2 Sophus::SE3d T_prev = T_cw;      // last accepted pose
3
4 for (int iter = 0; iter < maxIterations; ++iter)
5 {
6     // ... compute H, b, new_cost at T_cw ...
7
8     Sophus::SE3d T_new = Sophus::SE3d::exp(delta) * T_cw;
9
10    if (new_cost < cost)
11    {
12        T_prev = T_cw; // save current as "previous" before moving
13        T_cw = T_new;
14        cost = new_cost;
15        lambda /= 10.0;
16    }
17    else
18    {
19        T_cw = T_prev; // roll back to last accepted pose
20        lambda *= 10.0;
21    }
22 }
```

Listing 4: Standard LM accept/reject with rollback — `solvePnPWeighted()`

The rollback means the optimizer is always operating from a pose it has *confirmed* to be better than its predecessor. With a large λ , the step is small and the comparison is strict, so the loop converges quickly from any accepted pose.

3.3 Bug 3 — Geometric degeneracy guard breaks out without rollback

Inside the inner loop, the code checked `valid_pts` (the number of points that project in front of the camera, $Z_c > 0$). If `valid_pts < 4`, it originally executed `break`. After the first forced step commits T_1 , if T_1 sends the camera behind most scene points, `valid_pts < 4` would fire at $\text{iter}=1$ and break with $T_{\text{cw}} = T_1$ as the final pose.

The fix saves $T_{\text{last_good}}$ — the most recent pose at which `valid_pts  $\geq 4$` was confirmed — and restores it before breaking:

```

1 Sophus::SE3d T_last_good = T_cw; // last pose with valid_pts >= 4
2
3 for (int iter = 0; iter < maxIterations; ++iter)
4 {
5     int valid_pts = 0;
```

```

6   for (int i = 0; i < n; ++i)
7   {
8       Eigen::Vector3d Pc = T_cw * Xw_i;
9       if (Pc(2) <= 0) continue;
10      // ... accumulate H, b, new_cost ...
11      ++valid_pts;
12  }
13
14  if (valid_pts < 4)
15  {
16      T_cw = T_last_good;    // restore last geometrically valid pose
17      break;
18  }
19  T_last_good = T_cw;    // current T_cw confirmed valid; save it
20
21  // ... solve delta, accept/reject ...
22  }

```

Listing 5: Geometric degeneracy guard with rollback — solvePnPWeighted()

3.4 Final fallback — bad local minimum

Even with the rollback fixes, the LM can converge to a local minimum of the *total* weighted squared reprojection error where the RANSAC inlier set has fewer than 4 points within the 8 px threshold. This is mathematically valid (the optimizer found a lower cost), but useless for localization.

The fallback reclassifies inliers at T_{init} (the RANSAC pose) instead of the converged pose. Because all input correspondences are RANSAC inliers by construction, they are guaranteed to be within 8 px at T_{init} :

```

1  // After the LM loop, classify inliers at the converged T_cw.
2  std::vector<int> inliers;
3  for (int i = 0; i < n; ++i)
4  {
5      // ... project at T_cw, check < inlierThresholdPx ...
6      if (reprojError < inlierThresholdPx)
7          inliers.push_back(i);
8  }
9
10 if ((int)inliers.size() < 4)
11 {
12     // LM converged to a bad local minimum -- fall back to the initial
13     // (RANSAC) pose, which is guaranteed to have all input points as
14     // inliers.
15     inliers.clear();
16     for (int i = 0; i < n; ++i)
17     {
18         Eigen::Vector3d Pc = T_init * Xw_i;
19         if (Pc(2) <= 0) continue;
20         if (reprojError_at_T_init < inlierThresholdPx)
21             inliers.push_back(i);
22     }
23     if ((int)inliers.size() < 4)
24         return result;    // genuine failure (T_init itself is degenerate)
25
26     // Overwrite rvec/tvec with T_init before returning

```

```

26 // (use T_init rotation/translation for the result)
27 }

```

Listing 6: `T_init` fallback when LM converges to a bad local minimum — `solvePnPWeighted()`

Note

The fallback returns the RANSAC pose when LM fails, not a failure code. The coverage improvement comes from this: frames where LM diverges now contribute a valid (if unrefined) pose rather than being dropped from the comparison trace.

§4 Summary of All Changes

Table 1: Complete list of changes, files modified, and their purpose

Change	File	Purpose
Add uniform LM after <code>solvePnP()</code> in <code>processFrame()</code>	<code>relocalization.cpp</code>	Standard PnP baseline now uses RANSAC+EPnP+LM, matching WeightedPnP algorithm
Remove <code>!landmarkRegions.empty()</code> guard	<code>relocalization_node.cpp</code>	WeightedPnP runs on every localized frame, not only landmark frames
Inlier threshold: <code>mMinInliers(15)</code> → 4	<code>relocalization.cpp</code>	LM result not discarded for having “few” inliers vs RANSAC minimum
Add <code>T_last_good</code> rollback on <code>valid_pts < 4</code>	<code>relocalization.cpp</code>	Geometric degeneracy no longer leaves optimizer at a behind-camera pose
Add <code>T_prev</code> rollback on rejected step	<code>relocalization.cpp</code>	Standard LM behaviour: bad steps roll back to last accepted pose
Add <code>T_init</code> fallback after bad local minimum	<code>relocalization.cpp</code>	LM failure returns RANSAC pose instead of dropping the frame

§5 Results

Verification was performed on the TUM FR3 long-office-household validation sequence (657 frames). The table below shows frame coverage across development iterations:

Table 2: WeightedPnP coverage improvement over development iterations (TUM FR3 validation sequence, `bg_weight=1.0`)

State	Change applied	WPnP coverage
Baseline	<i>none — landmark guard active</i>	34 / 655 (5.2%)
After fix 1	Landmark guard removed	200 / 673 (29.7%)
After fix 2	Inlier threshold lowered: $15 \rightarrow 4$	389 / 657 (59.2%)
After fix 3	<code>T_last_good</code> rollback (valid_pts)	449 / 680 (66.0%)
After fix 4	<code>T_prev</code> rollback (LM rejection)	446 / 656 (68.0%)
After fix 5	<code>T_init</code> fallback (bad local min.)	644 / 665 (96.8%)

Note

The small frame-count variation across rows (655–680) reflects run-to-run variation in the BoW retrieval and RANSAC, which are non-deterministic.

The remaining 21 failed frames all have `std_inliers` ≤ 5 . These reach the $n < 6$ guard inside `solvePnPWeighted` (a minimum for a well-conditioned LM system), so they are expected failures. No fix can recover them without more correspondences.

5.1 Pose agreement at `bg_weight=1.0`

On all 644 successful frames, the pose delta between standard PnP and WeightedPnP is exactly 0.000 m:

Table 3: Pose delta (WeightedPnP vs Standard PnP) on successful frames, `bg_weight=1.0`, `lm_weight=1.0`

Metric	Mean	Median	Max	% under 1 mm
Pose delta (m)	0.0000	0.0000	0.0000	100%

This confirms the invariant stated in §2: with uniform weights, both paths execute the same computation on the same data and produce identical results.

5.2 Interpretation for the thesis

After this change, every comparison plot generated with `bg_weight=1.0` shows the same timestamp density and the same pose trace as the Standard PnP baseline. Any *difference* observed at other weight values (e.g. `bg_weight=0.3`) is attributable solely to the semantic weight vector and not to structural differences between the two pipelines. This makes the experimental comparison scientifically valid.

The Standard PnP baseline reported in all plots generated after this change is **RANSAC+EPnP+LM weights**), not raw RANSAC+EPnP. This should be stated explicitly in the experimental setup section of the thesis.

§6 Complete LM Loop After All Fixes

The final state of the Levenberg–Marquardt loop in `solvePnPWeighted()`, incorporating all three robustness fixes:

```

1 const Sophus::SE3d T_init      = T_cw; // RANSAC pose (warm start)
2 Sophus::SE3d      T_last_good = T_cw; // last pose with valid_pts >= 4
3 Sophus::SE3d      T_prev      = T_cw; // last accepted pose
4 double lambda = 1e-3;
5 double cost    = std::numeric_limits<double>::max();
6
7 for (int iter = 0; iter < maxIterations; ++iter)
8 {
9     Eigen::Matrix<double,6,6> H = Eigen::Matrix<double,6,6>::Zero();
10    Eigen::Matrix<double,6,1> b = Eigen::Matrix<double,6,1>::Zero();
11    double new_cost = 0.0;
12    int valid_pts = 0;
13
14    for (int i = 0; i < n; ++i)
15    {
16        Eigen::Vector3d Pc = T_cw * Xw_i;
17        if (Pc(2) <= 0) continue;
18        // accumulate H, b, new_cost ...
19        ++valid_pts;
20    }
21
22    // Fix B3: geometric degeneracy guard with rollback
23    if (valid_pts < 4)
24    {
25        T_cw = T_last_good;
26        break;
27    }
28    T_last_good = T_cw; // confirmed valid
29
30    // LM damping
31    Eigen::Matrix<double,6,6> H_damped = H;
32    for (int k = 0; k < 6; ++k)
33        H_damped(k,k) += lambda * std::max(H(k,k), 1e-8);
34
35    Eigen::Matrix<double,6,1> delta = H_damped.ldlt().solve(b);
36    if (!delta.allFinite()) { lambda *= 10.0; result.iterations = iter
37        +1; continue; }
38
39    Sophus::SE3d T_new = Sophus::SE3d::exp(delta) * T_cw;
40
41    // Fix B2: standard LM accept/reject with rollback
42    if (new_cost < cost)
43    {
44        T_prev = T_cw;
45        T_cw    = T_new;
46        cost    = new_cost;
47        lambda /= 10.0;
48    }
49    else
50    {
51        T_cw = T_prev; // roll back
52        lambda *= 10.0;
53    }
54 }

```

```
53     result.iterations = iter + 1;
54     if (delta.norm() < convergenceThreshold) break;
55 }
56
57 // Classify inliers at converged T_cw
58 // Fix B1: threshold is 4, not mMinInliers
59 std::vector<int> inliers;
60 // ... classify at T_cw ...
61
62 // Fix B4: T_init fallback when LM found a bad local minimum
63 if ((int)inliers.size() < 4)
64 {
65     inliers.clear();
66     // ... reclassify at T_init ...
67     if ((int)inliers.size() < 4) return result; // genuine failure
68     // overwrite rvec/tvec with T_init before returning
69 }
70 }
```

Listing 7: Complete LM loop with all robustness fixes — `relocalization.cpp`